Credit Card Fraud Detection

- Aaryaman Kompella (115447503) &
Yatharth Bahl (115053960)

Team Distribution

Aaryaman Kompella

- Took the lead in EDA
- Helped in refining the model
- Helped in gathering data

Yatharth Bahl

- Took the lead in Modeling
- Helped in EDA
- Helped in gathering data

Agenda

1. Exploratory Data Analysis
2. Modeling
3. Conclusion

Checklist for EDA

1

Data Cleaning

Ensure data is free of excessive NaN values, checking for outliers, ensuring columns are stored in correct data types, etc.

2

Heatmap

Construct and analyze heatmap to quantify the correlation between columns in data, useful for further analysis.

3

Scatterplot

Helps visualize relationships between variables, detect outliers and understand the concentration of data.

4

Boxplot

Summarizes data for specific variables and helping in narrowing focus.

5

Probability & Bar plot

Shows the distribution of specific variable on a curve, and helps compare importance of variables.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 8 columns):
 #   Column                          Non-Null Count    Dtype  
---  ------                          --------------    -----  
 0   distance_from_home              1000000 non-null  float64
 1   distance_from_last_transaction  1000000 non-null  float64
 2   ratio_to_median_purchase_price  1000000 non-null  float64
 3   repeat_retailer                 1000000 non-null  float64
 4   used_chip                       1000000 non-null  float64
 5   used_pin_number                 1000000 non-null  float64
 6   online_order                    1000000 non-null  float64
 7   fraud                           1000000 non-null  float64
dtypes: float64(8)
memory usage: 61.0 MB
```

```
#Missing values
print(df.isna().sum())
#No missing values
```

```
distance_from_home              0
distance_from_last_transaction  0
ratio_to_median_purchase_price  0
repeat_retailer                 0
used_chip                       0
used_pin_number                 0
online_order                    0
fraud                           0
dtype: int64
```

```
df=pd.read_csv('card_transdata.csv')
df.head()
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_purchase_price	repeat_retailer	used_chip	used_pin_number	online_order	fraud
0	57.877857	0.311140	1.945940	1.0	1.0	0.0	0.0	0.0
1	10.829943	0.175592	1.294219	1.0	0.0	0.0	0.0	0.0
2	5.091079	0.805153	0.427715	1.0	0.0	0.0	1.0	0.0
3	2.247564	5.600044	0.362663	1.0	1.0	0.0	1.0	0.0
4	44.190936	0.566486	2.222767	1.0	1.0	0.0	1.0	0.0

```
df.describe()
```

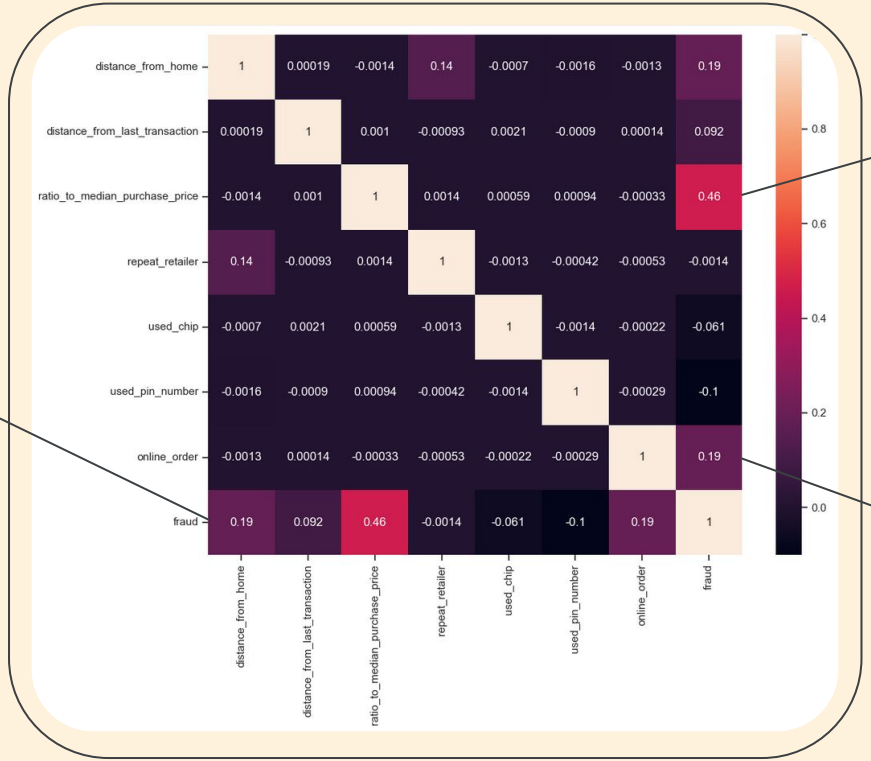
	distance_from_home	distance_from_last_transaction	ratio_to_median_purchase_price	repeat_retailer	used_chip	used_pin_number	online_order	fraud
count	1000000.000000	1000000.000000	1000000.000000	1000000.000000	1000000.000000	1000000.000000	1000000.000000	1000000.000000
mean	26.628792	5.036519	1.824182	0.881536	0.350399	0.100608	0.650552	0.087403
std	65.390784	25.843093	2.799589	0.323157	0.477095	0.300809	0.476796	0.282425
min	0.004874	0.000118	0.004399	0.000000	0.000000	0.000000	0.000000	0.000000
25%	3.878008	0.296671	0.475673	1.000000	0.000000	0.000000	0.000000	0.000000
50%	9.967760	0.998650	0.997717	1.000000	0.000000	0.000000	1.000000	0.000000
75%	25.743985	3.355748	2.096370	1.000000	1.000000	0.000000	1.000000	0.000000
max	10632.723672	11851.104565	267.802942	1.000000	1.000000	1.000000	1.000000	1.000000

Data Cleaning

As dataset was pulled from kaggle, there were no missing values. Data was readable and stored in correct formats. There was a wide variety ensuring no bias.

Heatmap

There were 3 main observations from the heatmap, which gave a direction to continue exploratory analysis.



Fraud v/s Distance from home

The distance from home for a transaction also had a strong correlation with fraud.

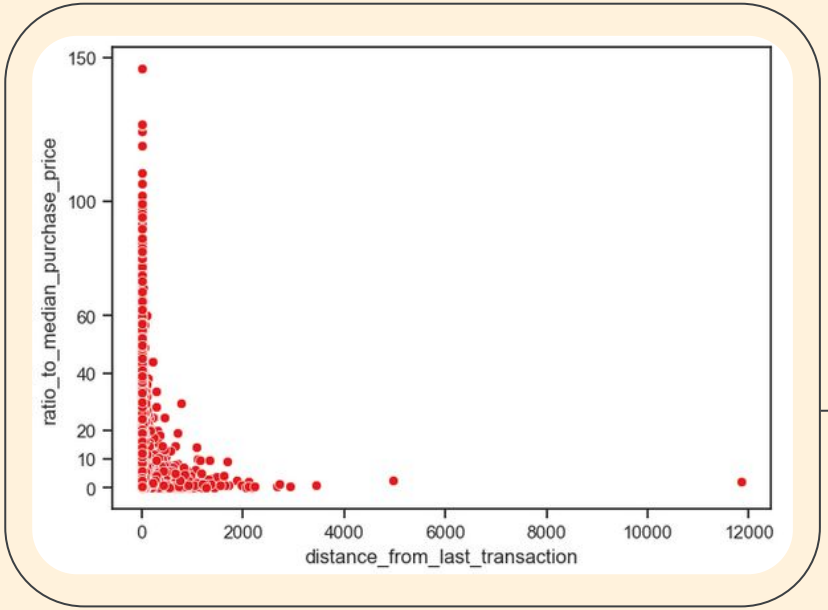
Ratio v/s Fraud

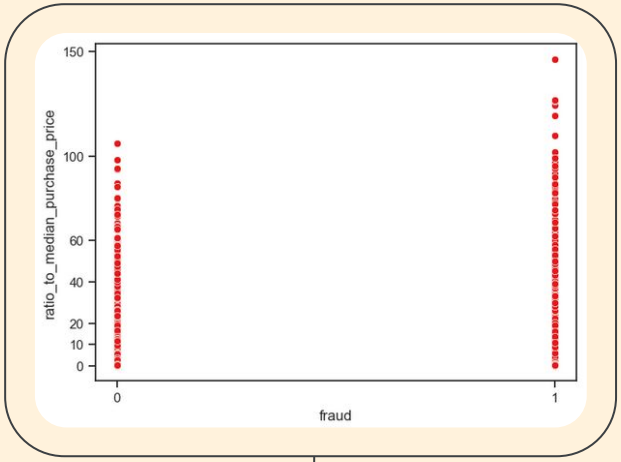
The ratio of median purchase price (RMPP) and Fraud had the strongest correlation.

Online order v/s Fraud

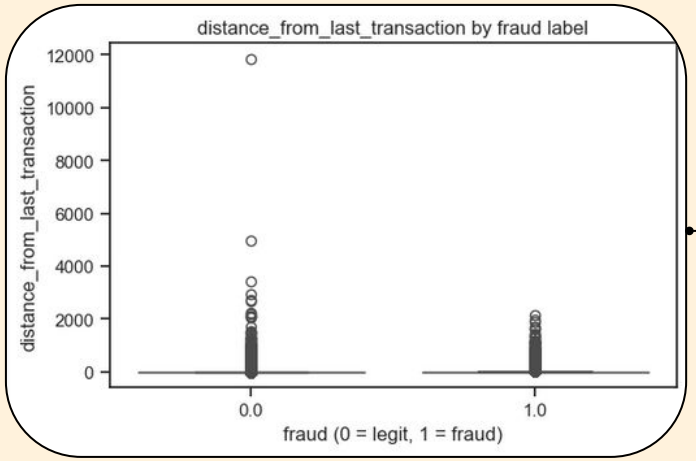
The correlation between online order and fraud was also very strong.

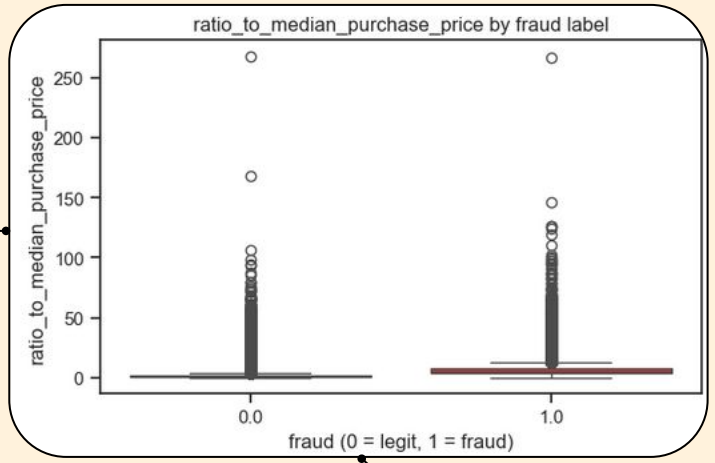
Scatterplot



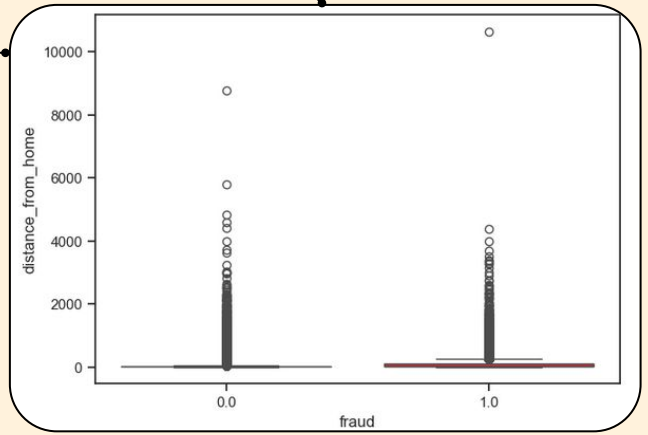


- Ratio of purchase price is highly concentrated below 60, which makes sense as common purchases range from $5 coffees to $300 groceries, which is the suitable explanation for recurring purchases within 2000 km from last transaction.
- The transactions with a RMPP greater than 60 are more likely to be a fraudulent one, which makes sense as the goal is to use up all the money as fast as possible.





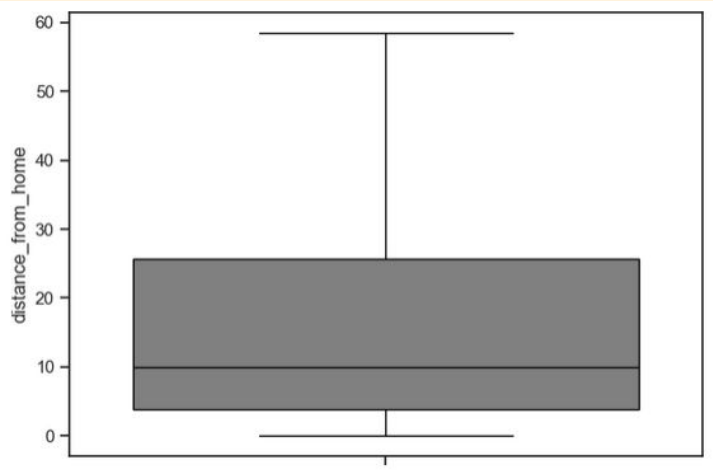
Boxplot



The main takeaways are:-

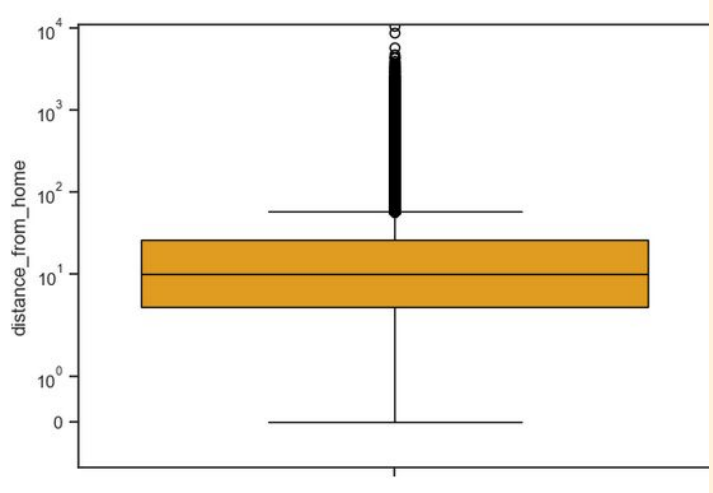
1. Fraud transactions are more likely to occur farther away from home address, in the range of 1500 and 3500 km.
2. The RMPP for fraudulent transactions is clustered between 15 and 125.
3. As fraudulent transactions tend to be repeated at same stores, places etc., the distance between each transaction tends to stay below 2000 km.

Box Plots (continued)



Distance from Home (Linear)

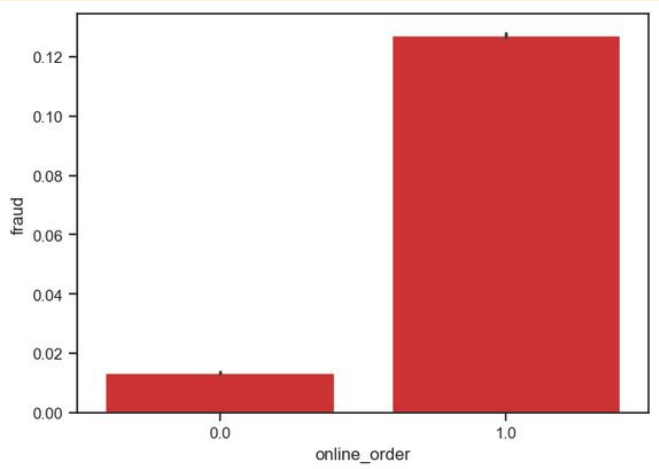
On a linear box plot of distance from home, with the points removed, we see the average distribution of data being concentrated between 5 and 25. The median is 10, which explains that most transactions tend to occur in an approximately 30 km radius around a user's' home address.



Distance from Home (Logarithmic)

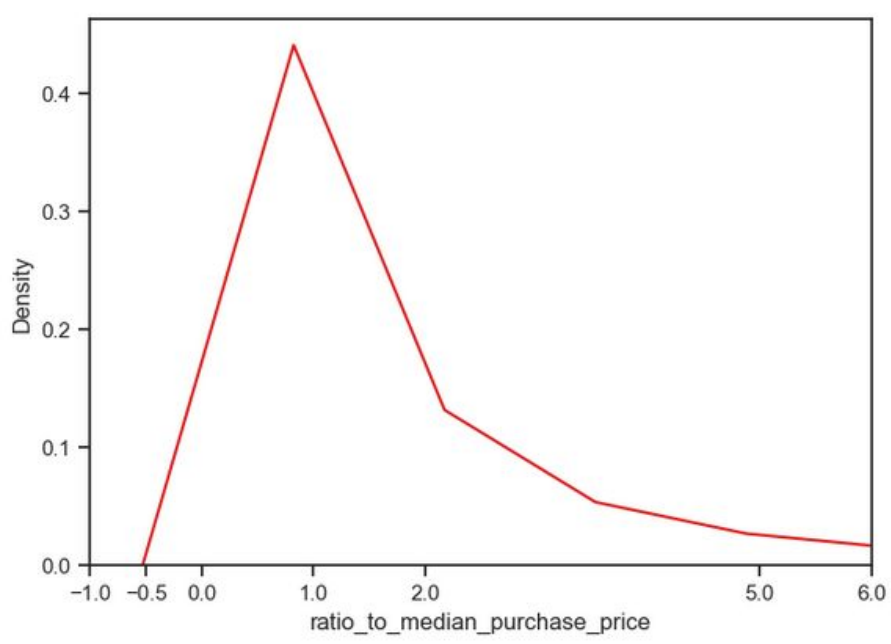
On a logarithmic scale of box plot, with the points still visible, we see that while the average distribution is in the 10^1 range, there are many outliers. There are a lot of outliers that fall in the range of 10^2, 10^3 and a few in 10^4. This explains transactions that occur in neighbouring cities or states, with a few on the higher end being international trips.

Probability and Bar Plot



We can infer from these plots that :-

1. **Bar Plot** - Online orders make up 13% of transactions, which are mostly fraudulent.
2. **Probability plot** - The ratio to median purchase price tends to be concentrated between -0.5 and 5.



Modelling

DECISION TREE CLASSIFIER

```
from sklearn.tree import
 DecisionTreeClassifier
```



Train–Test Split & Preprocessing

No missing values and all columns are already numeric, so preprocessing is minimal.

- We use the original features directly (no scaling needed for decision trees).
- Split the data into:
  - **67% training** (670,000 rows)
  - **33% testing** (330,000 rows)
- This keeps enough data to both **train** and **reliably evaluate** the model.

CODE:

```
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.33,
random_state=42)
```

Model Choice – Decision Tree

Chosen model: DecisionTreeClassifier (scikit-learn).

Reasons for using a decision tree:

- Handles non-linear relationships and feature interactions (e.g., “large distance from home AND high amount” → fraud).
- Works directly with unscaled numeric features.
- Is interpretable: we can explain decisions in terms of simple rules.

CODE:

```
clf = DecisionTreeClassifier(random_state=0)
X=df.drop('fraud', axis=1)
y = df['fraud')
```

.

Handling Class Imbalance

- Only 8.7% of the data is fraud i.e. model could easily be biased toward predicting “not fraud” all the time.
- To handle this, we set class_weight = "balanced" in the decision tree:
    - This gives more weight to fraud cases during training.
- For model selection we optimize F1-score, not just accuracy:
    - F1 balances precision (how many predicted frauds are actually fraud) and recall (how many real frauds we catch).

CODE:

```
params = {
    'criterion': ["gini", "entropy"], # log_loss is rarely necessary for simple trees
    'max_depth': [3, 4, 5, 6],        # Prevent deep trees
    'min_samples_split': [20, 40, 60], # Require more data to split
    'min_samples_leaf': [2, 10, 20, 30], # Prevent tiny leaf nodes
    'max_features': ["sqrt", "log2"], # Force tree to look at subsets of features
    'class_weight': ["balanced"]      # Crucial for Fraud data (imbalanced)
}
```

Hyperparameter Tuning (GridSearchCV)

- A decision tree has many settings (depth, min samples, etc.), and guessing them can cause underfitting or overfitting.
- GridSearchCV tests many hyperparameter combinations and, with cross-validation, picks the one that performs best on unseen data.
- Here we used 3-fold CV, kept class_weight="balanced" for the imbalanced labels, and chose the model that maximized F1 for the fraud class (good balance of precision and recall).
- The selected parameters give us a tree that captures fraud patterns well while still generalizing to new transactions.

CODE:

```
gm=GridSearchCV(
    estimator=clf,
    param_grid=params,
    scoring='f1',
    cv=3,
    n_jobs=1,
    verbose=2
)
```

```
gm.best_params_
```

```
[21]:

{'class_weight': 'balanced',
 'criterion': 'entropy',
 'max_depth': 6,
 'max_features': 'sqrt',
 'min_samples_leaf': 20,
 'min_samples_split': 20}
```

Model Performance – Train vs Test

- On the train set (670,000 rows), the model reaches an accuracy of **0.973**, with precision 0.89, recall 0.80, and an F1-score of 0.84 for the fraud class.
- On the test set (330,000 rows), the model again reaches an accuracy of **0.973**, with precision 0.89, recall 0.79, and an F1-score of 0.84 for the fraud class.
- These results show that the model's performance on the fraud class is almost identical on the train and test data.
- Because the metrics are so similar, we can conclude that the model is **not significantly overfitting and generalizes well to new transactions.**

Training and Testing classification report

Train accuracy: 0.9733761194029851

Train classification report:

	precision	recall	f1-score	support
0.0	0.98	0.99	0.99	611385
1.0	0.89	0.80	0.84	58615
accuracy			0.97	670000
macro avg	0.93	0.89	0.91	670000
weighted avg	0.97	0.97	0.97	670000

Test accuracy: 0.9730939393939394

Test classification report:

	precision	recall	f1-score	support
0.0	0.98	0.99	0.99	301212
1.0	0.89	0.79	0.84	28788
accuracy			0.97	330000
macro avg	0.93	0.89	0.91	330000
weighted avg	0.97	0.97	0.97	330000

Confusion Matrix – Test Set

Out of 330,000 test transactions:

- True negatives (legit correctly predicted): **298,269**
- False positives (legit flagged as fraud): **2,943**
- False negatives (fraud missed): **5,936**
- True positives (fraud correctly caught): **22,852**
- The model catches **around 79% of frauds**.
- It raises some false alarms, but in fraud detection, **missing fraud is usually more costly** than a few extra checks.

Confusion Matrix

```
confusion_matrix(y_test,y_pred)
```

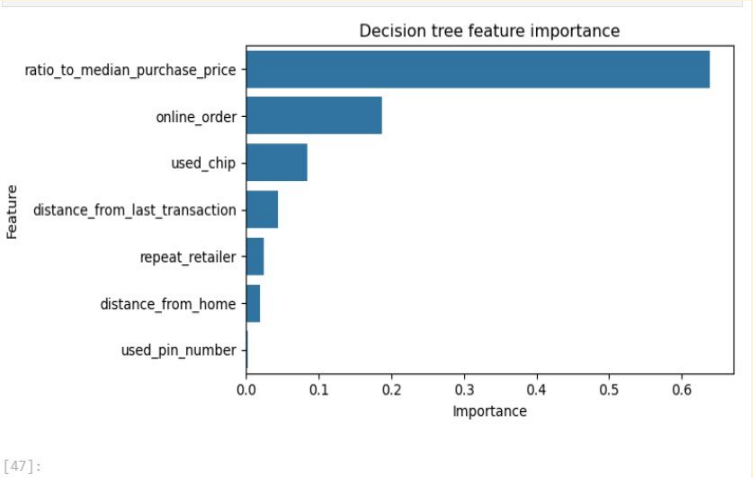
```
[27]:

array([[298269,   2943],
       [  5936,  22852]], dtype=int64)
```

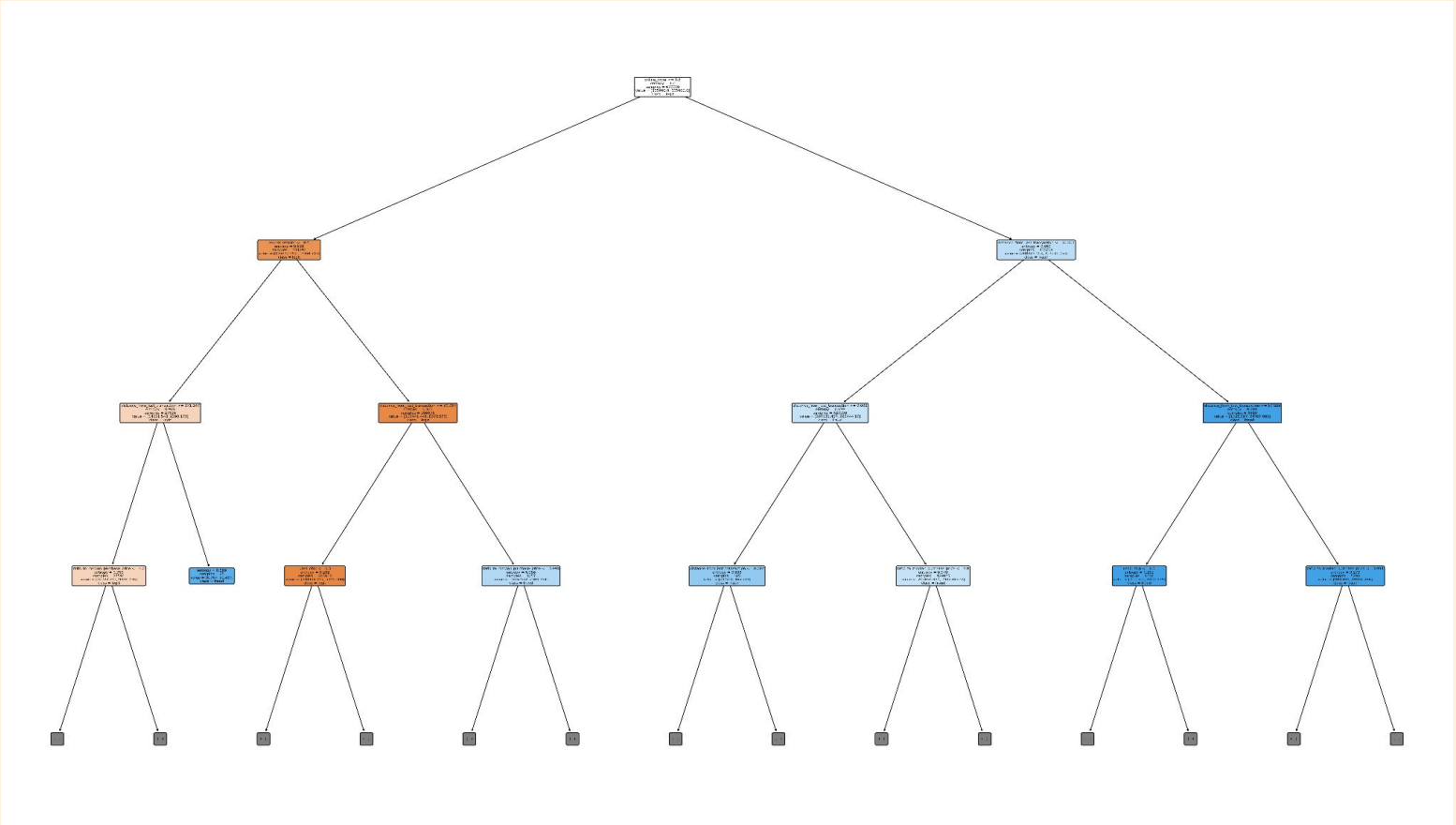
Our Findings: What drives the fraud prediction?

- This chart shows how important each feature is in the decision tree.
- **ratio_to_median_purchase_price** is by far the most important: transactions that look unusually expensive compared to the customer's normal spending strongly increase the chance of being flagged as fraud.
- **online_order** is the second most important factor, meaning online purchases are treated as riskier than in-store ones.
- **used_chip** and **distance_from_last_transaction** also play a role: card-present chip transactions and purchases close in time/location are generally safer.
- Features like **repeat_retailer**, **distance_from_home**, and **used_pin_number** matter less in this model, but they still add small extra signals on top of the main drivers
-

Feature Importance



Model Visualization





Ask us anything!